

An introduction to agent technology

H S Nwana and D T Ndumu

Intelligent agent technology is a rapidly developing area of research. However, in reality, there is a truly heterogeneous body of work being carried out under the 'agent' banner. In this paper, software agent technology is introduced by briefly overviewing the various agent types currently under investigation by researchers.

1. Introduction

The word 'agent' is currently in vogue in the popular computing press and within the artificial intelligence (AI) and computer science communities. It has become a buzzword because it is both a technical concept and a metaphor. However, its rampant use could conjure up the problems faced with other flamboyant titles including 'artificial intelligence' itself; far too ambitious claims precede the real technical work that follows.

This paper presents the real challenges and potential benefits of agent research. The main goal is to overview the rapidly evolving area of software agents; the overuse of the word 'agent' has tended to mask the fact that, in reality, there is a truly heterogeneous body of research being carried out under this banner. This paper places agents in context, defines them and then goes on to overview critically the rationales, hypotheses, goals, challenges and state-of-the-art demonstrators of the various agent types currently under investigation. It also proceeds to overview some other general issues which pertain to all the classes of agents identified.

2. Software agents — history and the context of this paper

Arguably, software agents date back to the early days of AI work, indeed, to Carl Hewitt's concurrent actor model [1]. In this model, Hewitt proposed the concept of a self-contained, interactive and concurrently-executing object which he termed an 'actor'. This object had some encapsulated internal state and could respond to messages from other similar objects.

Along with distributed problem solving and parallel AI, software agents and multiagent systems (MAS) form collectively one of the three broad areas which fall under distributed AI (DAI). Hence, they inherit many of DAI's

motivations, goals and potential benefits, for example, modularity (which reduces complexity), speed (due to parallelism), reliability (due to redundancy) and flexibility (i.e. new tasks are composed more easily from the more modular organisation). They also inherit benefits from AI such as operation at the knowledge level, easier maintenance, reusability and platform independence [2].

Broadly, for the purposes of this paper, the research on agents is split into two generations — the first spanning the period 1977—1990, and the second from 1990 to the current day. First generation work on agents concentrated mainly on deliberative-type agents with symbolic internal models¹. Particularly, they concentrated on macro issues such as the interaction and communication between agents, the decomposition and distribution of tasks, co-ordination and co-operation, and conflict resolution via negotiation. These 'macro' aspects of agents emphasise the society of agents over individual agents. The goal was to specify, analyse, design and integrate systems consisting of multiple collaborating agents. Chaib-draa et al [3], Bond and Gasser [4] and Gasser and Huhns [5] present excellent reviews of work on the first generation of agents. It is important to note that this work still progresses.

Second generation work on agents is viewed as being characterised by two major and distinct strands — research and development of agent theories, architectures and languages, and a significant broadening of the typology of agents being investigated. The former research is well summarised in Wooldridge and Jennings [6, 7] and Wooldridge et al [8]. This paper complements them by concentrating on overviewing the broadening typology of agents being investigated by agent researchers.

¹A deliberative agent is "one that possesses an explicitly represented, symbolic model of the world, and in which decisions (for example about what actions to perform) are made via symbolic reasoning" [9].

3. Agent Applications

The range of firms and universities actively pursuing agent technology is quite broad and the list is ever growing. It includes small non-household names, medium-size organisations and the large multinationals. The scope of the applications being developed is arguably more impressive — it really does range from the mundane to the moderately ‘smart’. Towards the smart end of the spectrum are the likes of Sycara’s visitor hosting system [10] at Carnegie Mellon University (CMU). In this system, agents co-operate in order to create and manage a visitor’s schedule to CMU.

To achieve this, the agents firstly access on-line information resources in order to determine the visitor’s name, organisation, status in their organisation, areas of interest and projects being worked on. Secondly, using the information gathered about the visitor, they retrieve information (e.g. rank, telephone number and e-mail address) from personnel databases in order to determine appropriate faculty members to meet the visitor. Thirdly, the visitor hosting agent composes messages which it despatches to the calendar agents of these faculty members, asking whether they are willing to meet this visitor and at what time. Next, the responses are collated and the visitor hosting agent creates the schedule for the visitor which involves booking rooms for the various appointments with faculty members. Naturally, the system interacts with the human organiser, seeking confirmation, refutations, suggestions and advice.

Application domains where agent solutions are being applied or researched include work-flow management, network management, air-traffic control, business process re-engineering, data mining, information retrieval/management, electronic commerce, education, personal digital assistants (PDAs), e-mail filtering, digital libraries, command and control, smart databases, scheduling/diary management. Indeed, as Guilfoyle notes:

“...in 10 years’ time most new IT development will be affected, and many consumer products will contain embedded agent-based systems.” [11]

4. What is an agent?

There is as much chance of agreeing on a consensus definition for the word ‘agent’ as there is of AI researchers arriving at one for ‘artificial intelligence’! When necessary an agent is defined as referring to a component of software and/or hardware which is capable of acting exactly in order to accomplish tasks on behalf of its user. However, it would be preferable to say it is an umbrella term which covers a range of other more specific agent types, and then go on to list and define what these other agent types are.

4.1 A typology of agents

There are several dimensions to classifying existing software agents. Firstly, agents may be classified by their mobility, i.e. by their ability to move around some network — this yields the classes of static or mobile agents. Secondly, they may be classed as either deliberative or reactive. Deliberative agents derive from the deliberative thinking paradigm which holds that agents possess an internal symbolic reasoning model, and they engage in planning and negotiation with other agents in order to achieve their goals. Work on reactive agents originates from research carried out by Brooks [12] and Agre and Chapman [13]. These agents do not have any internal symbolic models of their environment, and they act using a stimulus/response type of behaviour by responding to the present state of the environment in which they are embedded [14].

Thirdly, agents may be classified along several attributes which ideally they should exhibit. At BT Laboratories, a minimal list of three has been identified — autonomy, learning and co-operation. Autonomy refers to the principle that agents can operate on their own without the need for human guidance, even though this would sometimes be invaluable. A key element of autonomy is proactiveness, i.e. the ability to ‘take the initiative’ [6]. Co-operation with other agents is paramount — it is the *raison d’être* for having multiple agents. Further, the communication required to ensure co-operation generally involves high-level messages. The use of high-level messaging leads to lower communications costs, easy re-implementability, and concurrency. Lastly, for agents to be truly ‘smart’, they would have to learn as they react and/or interact with their external environment, so that, with time, their performance increases. These three characteristics are used in Fig 1 to derive three types of agent to include in this typology — collaborative agents, interface agents and truly smart agents.

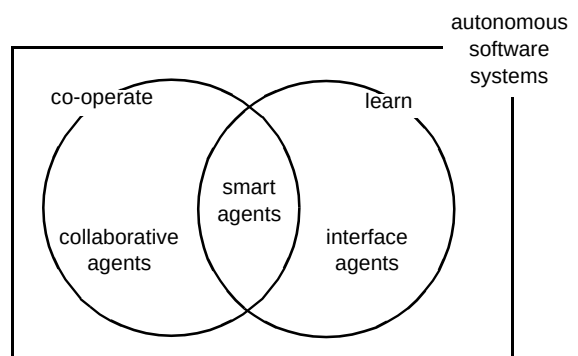


Fig 1 A part view of an agent typology.

It is emphasised that these distinctions are not definitive. For example, with collaborative agents, there is more

emphasis on co-operation and autonomy than on learning; but it is not implied that collaborative agents never learn. Anything else which lies outside the ‘circles’ is not considered to be an agent. For example, most expert systems are largely autonomous; but, typically, they do not co-operate or learn. Ideally, agents should do all three equally well, but this is the aspiration rather than the reality. Truly smart agents do not yet exist.

Fourthly, agents may sometimes be classified by their roles (particularly, if the roles are major ones), e.g. World Wide Web (WWW) information-gathering agents. Essentially, such agents help manage the vast amount of information in wide area networks like the Internet. This class of agent is referred to as an information or Internet agent. Again, information agents may be static or mobile and deliberative or reactive.

Fifthly, also included is the category of hybrid agents which combine two or more agent philosophies in a single agent.

There are other attributes of agents which are considered secondary to those already mentioned. For example, is an agent versatile (i.e. does it have many goals or does it engage in a variety of tasks)? Is an agent benevolent or non-helpful, antagonistic or altruistic? Does an agent lie knowingly or is it always truthful? Is it temporally continuous? Perhaps unbelievably, some researchers are also attributing emotional attitudes to agents — do they get ‘fed up’ being asked to do the same thing time and time again [15]? Some agents are also imbued with mentalistic attitudes such as beliefs, desires and intentions [16, 17]. Such attributes as these provide for a stronger definition of agenthood.

In essence, agents exist in a truly multi-dimensional space. However, for the sake of clarity of understanding, this space has been ‘collapsed’ into a single list of seven types of agent:

- collaborative,
- interface,
- mobile,
- information/Internet,
- reactive,
- hybrid,
- smart.

There are some applications which combine agents from two or more of these categories, and these are referred to as heterogeneous agent systems.

5. A panoramic overview of the different agent types

This section contains an overview of all the types of agents identified in the typology of the previous section in terms of some or all of the following — their essential metaphors, hypotheses/goals, motivations, roles, prototypical examples, potential benefits, and key challenges. This overview does not include smart agents on the grounds that this is the aspiration of agent researchers rather than the reality.

5.1 Collaborative agents — an overview

As shown in Fig 1, collaborative agents emphasise autonomy and co-operation with other agents in order to perform tasks for their owners in open and time-constrained multi-agent environments. They may learn, but this aspect is not typically a major emphasis of their operation, though some perform limited parametric or learning by rote. To co-ordinate their activities, they may have to negotiate in order to reach mutually acceptable agreements. Collaborative agents tend to be static, large, coarse-grained agents. They may be benevolent, rational, truthful, some combination of these or neither. Most of the work classified in this paper as first generation investigated this class of agent. As noted earlier, some researchers are providing stronger definitions to such agents, and, as a result, the class of collaborative agents may itself be perceived as a broad grouping.

5.1.1 Hypothesis, motivation and benefits

The rationale for having collaborative agent systems is a specification of the goal of DAI. It may be stated as: “...creating a system that interconnects separately developed collaborative agents, thus enabling the ensemble to function beyond the capabilities of any of its members” [2]. Some other motivations for DAI research, and hence collaborative agent research, not already mentioned, include:

- solving problems that are too large for a centralised single agent to do due to resource limitations or the sheer risk of having one centralised system,
- allowing for the interconnecting and interoperation of existing legacy systems, e.g. expert systems, decision support systems, conventional programs, etc,
- providing solutions to inherently distributed problems, such as solutions which draw from distributed information sources such as distributed on-line information sources or distributed sensor networks (e.g. DVMT [18]), and solutions where the expertise is distributed, such as in health-care provisioning or air-traffic control (e.g. OASIS [17]).

5.1.2 A prototypical example — the Pleiades system

The Pleiades project [19] applies collaborative agents in the domain of organisational decision making over the 'infosphere' (which refers essentially to a collection of Internet-based heterogeneous resources).

Pleiades is a distributed collaborative agent-based architecture which has two layers of abstraction — the first layer contains task-specific collaborative agents and the second information-specific collaborative agents (see Fig 2). This architecture was used to develop the visitor hosting system which was described earlier. Task-specific agents (TA) perform a particular task for their users, e.g. arranging appointments and meetings with other task agents. These agents co-ordinate and schedule plans based on the context. They collaborate with one another (within layer 1) in order to resolve conflicts or integrate information. In order to garner the information required at this level, they request information from information-specific agents (IA). Information agents, in turn, may collaborate with one another (i.e. within layer 2) in order to provide the information requested back to the layer 1 requesting agent. The sources of the information are the many databases (DB) in the infosphere. Ultimately, the task agents propose a solution to their users.

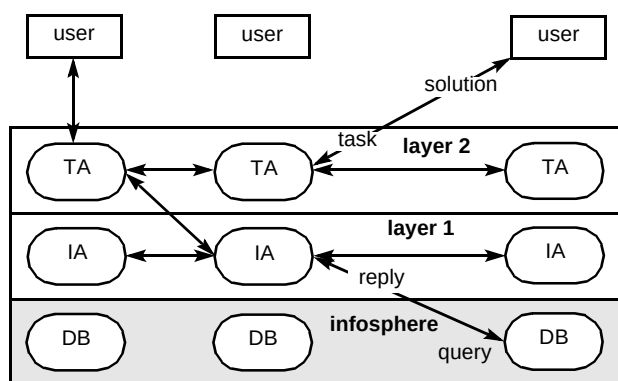


Fig 2 The Pleiades distributed system architecture (adapted from Sycara [10]).

Task agents encode a model of the task domain and knowledge of how to perform tasks, as well as an acquaintance model detailing the capabilities of other task or information agents. They also possess some learning mechanisms. Information agents possess knowledge of the various information sources and how to access them, and an acquaintance model specifying the abilities of other information agents [10]. Individually, an agent consists of a planning module linked to its local beliefs and facts database. It also has a local scheduler, a co-ordination module and an execution monitor. Thus, agents can instantiate task plans, co-ordinate these plans with other agents and schedule/monitor the execution of their local actions. Interestingly, the architecture has no central planner and hence agents must all engage in co-ordination by

communicating to others their constraints, expectations and other relevant information.

5.1.3 A brief critical review of collaborative agent systems work

There are many other useful pieces of work on collaborative agents. At BT Laboratories, two prototype collaborative agent-based systems, ADEPT and MII, have been developed recently. ADEPT [20] employs collaborative agents in the application area of business process re-engineering while MII [21] uses collaborative agents to perform decentralised management and control of consumer electronics, typically PDAs or PCs integrated with services provided by the network operator.

The key criticism of collaborative agents levelled by some researchers stems from their grounding in the deliberative thinking paradigm. Researchers in the reactive agents camp argue that this results in brittle and inflexible demonstrators with slow response times. In section 5.5 the deliberative versus reactive agents debate is briefly discussed.

5.1.4 Collaborative agents — some key challenges

Despite successful demonstrators like the Pleiades system and ADEPT, collaborative agents have been deployed in few real industrial settings, e.g. the ARCHON project [22]. There are still many teething problems.

- Engineering the construction of collaborative agent systems — to paraphrase BT's Professor Robin Smith: 'We must move away from point solutions to point problems, and design methodologies which allow for quicker, more structured implementation of collaborative agent-based systems'.
- Inter-agent co-ordination — co-ordination is essential to enable groups of agents to solve problems effectively because of the constraints of resource boundedness and time. Without a clear theory of co-ordination, anarchy or deadlock can set in easily in collaborative agent systems. Much work is required to address the issues of co-ordination and negotiation.
- Stability, scalability and performance issues — investigations need to be carried out to establish suitable minimum levels of performance and, clearly, these systems have to be shown to be stable.
- Learning — what are the appropriate learning mechanisms for different types of problems? Would learning not lead to instability? How do you ensure an agent does not spend much of its time learning, instead of participating in its set-up?

- Evaluation of collaborative agent systems — how are they verified and validated to ensure they meet their functional specifications? Are unanticipated events handled properly?

In conclusion, despite the criticisms of collaborative agents, there are many industrial applications which would benefit significantly from them. For example, a potential major role is seen for them in business process management.

5.2 Interface agents — an overview

Interface agents (see Fig 1) emphasise autonomy and learning in order to perform tasks for their owners. Maes [23] points out that the key metaphor underlying interface agents is that of a personal assistant who is collaborating with the user in the same work environment. Note the subtle emphasis and distinction between collaborating with the user and collaborating with other agents as is the case with collaborative agents.

Essentially, interface agents support and provide proactive assistance, typically to a user learning to use a particular application such as a spreadsheet or an operating system. The agent observes and monitors the actions taken by the user in the interface (i.e. ‘watches over the shoulder of its user’), learns new ‘short-cuts’, and suggests better ways of doing the task. As for learning, typically, interface agents learn to assist their users in the following four ways:

- by observing and imitating the user,
- through receiving positive and negative feedback from the user,
- by receiving explicit instructions from the user,
- by asking other agents for advice.

Generally, the learning modes are memory-based learning by rote or parametric, though other techniques such as evolutionary learning are also being introduced. Their co-operation with other agents, if any, is limited to asking for advice.

Maes [24] specifies two preconditions to be fulfilled by suitable application programs for interface agents — firstly, that there is substantial repetitive behaviour in using the application (otherwise, the agent will not be able to learn anything), and, secondly, that this repetitive behaviour is potentially different for different users (otherwise, use a knowledge-based approach).

5.2.1 Benefits/roles

The general benefits of interface agents are threefold. Firstly, they make less work for the end user and application developer. Secondly, the agent can adapt, over time, to its user’s preferences and habits. Finally, know-how among the different users in a community may be shared (e.g. when agents learn from their peers). Perhaps these will be understood better by discussing a few of the roles for which Maes and her team at Massachusetts Institute of Technology (MIT) are building interface agents.

Kozierok and Maes [25] describe an interface agent, Calendar Agent, that assists its user in scheduling meetings. It can learn, over time, the preferences and commitments of its user, e.g. does not like to attend meetings on a Friday, prefers meetings in the morning. The learning techniques employed are memory-based learning and reinforcement learning. Dent et al [26] also describe a similar learning apprentice agent, the Calendar Apprentice (CAP).

Liebermann [27] describes Letizia, a keyword and heuristic-based search agent, which assists in Web browsing. Since most browsers encourage depth-first browsing, Letizia conducts a breadth-first search concurrently for other useful locations in which the user may be interested. It does this by ‘guessing’ the user’s intention from their browsing behaviour (e.g. keeps returning to some particular page) and proceeding to search using the search engine. By doing this, it is able to recommend some other useful serendipitous locations.

Maes [23] describes a news filtering agent, NewT, that helps users filter and select articles from a continuous stream of Usenet Netnews. NewT agents are trained by presenting to them positive and negative examples of what should or should not be retrieved. They are message-content and keyword-based but also exploit other information such as the author and source.

5.2.2 Interface agents — some challenges

The key criticism of interface agents is that, so far, they tend to function in stand-alone fashions or, at the most, only engage in restricted and task-specific communication with identical peers [28]. This is not necessarily bad but it would be useful to have interface agents being able to negotiate with their peers as with collaborative agents. Furthermore, as Mitchell et al note:

“...it remains to be demonstrated that knowledge learned by systems like CAP can be used to significantly reduce their users’ workload.” [29]

Some other challenges for interface agents include:

- carrying out experiments using various machine learning techniques over several domains to determine which learning techniques are preferable for what domains and why,
- guaranteeing the users' privacy and the legal quagmire which may ensue following the fielding of such agents [23],
- extending the range of applications of interface agents into other areas.

However, having stated these, there is no denying the fact that interface agents can be deployed in real applications in the short term because they are simple, operate in limited domains and require no co-operation.

5.3 Mobile agents — an overview

5.3.1 Hypothesis, motivation and benefits

Mobile agents are software processes capable of roaming wide area networks (WANs) such as the WWW, interacting with foreign hosts, gathering information on behalf of their owners and coming 'back home' having performed the duties set them. These duties may range from making a flight reservation to managing a telecommunications network. Mobile agents are agents because they are autonomous and they co-operate, albeit differently from collaborative agents. For example, they may co-operate or communicate by one agent making the location of some of its internal objects and methods known to other agents.

The key hypothesis underlying mobile agents is the idea that, in certain applications, they provide a number of practical, though non-functional, advantages which escape their static counterparts. For example, as BT's Barry Crabtree notes: "Imagine having to download many images just to pick out one. Is it not more natural to get your agent to 'go' to that location, do a local search and only transfer the chosen compressed image back across the network?"

5.3.2 How mobile agents work — a brief Telescript view

Telescript is an interpreted object-oriented and remote programming language which allows for the development of distributed applications [30]. Figure 3 summarises a part view of the Telescript architecture. The Telescript Development Environment (TDE) comprises, among other

things, the engine (interpreter and run-time development environment), browser, debugger and associated libraries.

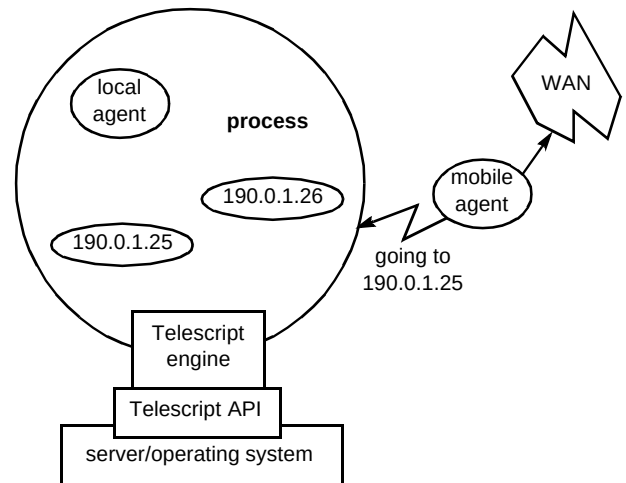


Fig 3 Part view of Telescript architecture (adapted from Wayner [31]).

Telescript applications consist of Telescript agents operating within a 'world' or cyberspace of places and engines; both of which are objects. The top class in Telescript's object hierarchy is the process. A Telescript engine is itself a pre-emptive multi-tasking interpreter which can run multiple processes. Hence, the engine can host multiple agents that share data/information between themselves. Furthermore, a place is a process which can contain an arbitrary number and depth of other places. Agents, unlike places, are objects which cannot contain other processes, but they can 'go' from place to place. An agent requiring a service defined at some given place must go to that place and call the operations there (see Fig 3). Thus, 'go' is the primitive which allows for inter-process communication. Two or more agent processes can meet in a place and make use of each other's services.

A 'go' requires a destination place and the host engine packages up the agent along with all its data, stack and instruction pointer and ships it off to this destination place which may be across a vast WAN. At its destination, the other Telescript-enabled engine unpacks it, checks its authentication, so that it is then free to resume execution at this new place. When it finishes, it returns to its original host having performed the task required by its owner.

There are other languages which support mobile agent system development, notably Java from Sun Microsystems, though it is not a remote programming language like Telescript. It is also important to point out that mobile agent systems need not only be constructed using a remote programming system like Telescript. Wayner [31] shows examples of how mobile agents can be scripted in Xlisp. Other languages to consider include Agent-Tcl, Safe-Tcl and C/C++. Indeed, Appleby and Steward [32] prototyped

an award-winning C/C++ programmed mobile agent-based system for controlling telecommunications networks.

5.3.3 Mobile agent applications

Mobile agent applications do not currently abound but are likely to increase in the future. However, the first commercial application was Sony's Magic Link PDA or personal intelligent communicator [33]. Essentially, it assists in managing a user's e-mail, facsimile, telephone and pager as well as linking the user to Telescript-enabled messaging and communications services such as America Online and AT&T PersonaLink Services.

Plu [34] mentions that France Telecom has prototyped some services based on Telescript. In one of their demonstrators, mobile Telescript agents integrate railway ticketing and car renting services. IBM plans to launch their Communication System [35] which uses mobile agents for providing a communications super-service capable of routing and translating communications from one service and medium to another, e.g. mobile to desktop, PDA to facsimile, speech to text.

5.3.4 Mobile agents — some challenges

Wayner [36] lists the major challenges of mobile agent research and development.

- Transportation — how does an agent pack up and move from place to place?
- Authentication — how do you ensure the agent is who it says it is, and is representing who it claims to be representing?
- Secrecy — how do you ensure that your agents maintain your privacy? How do ensure others do not read your personal agent and execute it for their own gains? How do you ensure your agent is not killed and its contents 'core-dumped'?
- Security — how do you protect against viruses? How do you prevent an incoming agent from entering an endless loop and consuming all the CPU cycles?
- Cash — how will the agent pay for services? How do you ensure that it does not run amok and run up an outrageous bill on your behalf?

In addition to these are the following.

- Performance issues — what would be the effect of having hundreds, thousands or millions of such agents on a WAN?
- Interoperability/communication/brokering services — how do you provide brokering/directory type services for locating engines and/or specific services? How do

you execute an agent written in one agent language on an agent engine written in another language? How do you publish or subscribe to services, or support broadcasting necessary for some other co-ordination approaches?

Having listed some of the challenges of mobile agent research, it must be noted that some of them are already being addressed successfully in development environments like TDE using various techniques including the following:

- using ASCII-encoded, Safe-Tcl scripts or MIME-compatible e-mail messages for transportation,
- using public- and private-key digital signature technology for authentication, cash and secrecy,
- providing limited languages that will not allow an agent to write to memory, say, for security.

As a result, much software and hardware which exploit mobile agent-based services are currently in the pipeline.

5.4 Information/Internet agents — an overview

5.4.1 Hypothesis, motivation and benefits

Information agents perform the role of managing, manipulating or collating information from many distributed sources. The motivation for developing information agents is at least twofold. Firstly, there is simply a need for tools to manage the information explosion of the WWW. Everyone on the WWW would benefit from them in the same way as they are benefiting from search facilitators such as spiders, lycos or webcrawlers. Secondly, there are vast financial benefits to be gained. Recall that Netscape Corporation grew from relative obscurity to a billion dollar company almost overnight and a Netscape or Mosaic client simply offers general browsing capabilities, albeit with a few add-ons. Whoever builds the first usable Netscape equivalent of a proactive, dynamic, adaptive and co-operative WWW information manager is certain to reap enormous financial rewards.

5.4.2 How information agents work

Internet agents could be mobile; however, this is not the norm as yet. Typical static ones are embedded within an Internet browser and use a host of Internet management tools such as spiders and search engines in order to gather the information. Etzioni and Weld [37] describe a state-of-the-art Internet agent called the Internet 'softbot' (software robot). It allows a user to make a high-level menu-based request such as 'send the budget memos to Mitchell at CMU' and 'Get all of Ginsberg's technical reports that aren't stored locally', and the softbot is able to use search and inference knowledge to determine how to satisfy the

request in the Internet. In doing so, it is able to tolerate ambiguity, omissions and the inevitable errors in the user's request. Etzioni and Weld use a strong analogy to a real robot in order to describe their softbot's interface to the Internet. For example, they describe the softbot's effectors to include ftp, telnet, mail and numerous file manipulation commands including mv or compress. The sensors provide the softbot with information about the external world and they include Internet facilities such asarchie, gopher and netfind.

5.4.3 A brief critical review of information agents work

Information agents are expected to be a major growth area in the next couple of years. At BT Laboratories, Davies and Weeks [38] have designed and implemented the Jasper agent. Jasper agents work on behalf of a user or a community of users, and are able to store, retrieve, summarise and inform other agents of information useful to them found on the WWW. As a user works with their Jasper agent, a profile of their interests is built dynamically, based on keywords. A Jasper agent is able to suggest interesting WWW pages to a user by matching their profile with those of other users in the community. A successful match results in the user being told of other WWW pages that peers find 'interesting'.

The key problem with static information agents is in keeping their indexes up-to-date in the very dynamic WWW environment. For this reason, and for similar reasons mentioned in section 5.3, it is probable that the majority of future information agents will be of the mobile variety. They will be able to navigate the WWW and store its topology in a database, say, at their home site.

As regards the criticisms and challenges of information agents, they are essentially similar to those of either interface or mobile agents, depending on whether the information agent is static or mobile respectively.

5.5 Reactive software agents — an overview

Reactive agents represent a special category of agents which do not possess internal, symbolic models of their environments; instead they respond in a stimulus-response manner to the present state of the environment in which they are embedded. Reactive agents work dates to research such as Brooks [12] and Agre and Chapman [13], but many theories, architectures and languages for these sorts of agents have subsequently been developed.

Maes [39] highlights the three key ideas which underpin reactive agents. Firstly, 'emergent functionality' — reactive agents are relatively simple and they interact with other agents in basic ways. Nevertheless, complex patterns of behaviour emerge from these interactions when the

ensemble of agents is viewed globally. Hence, there is no *a priori* specification (or plan) of the behaviour of the set-up of reactive agents. Secondly, 'task decomposition' — a reactive agent is viewed as a collection of modules which operate autonomously and are responsible for specific tasks (e.g. sensing, motor control, computations, etc). Communication between the modules is minimised and of quite a low-level nature. No global model exists within any of the agents and, hence, the global behaviour has to emerge. Thirdly, reactive agents tend to operate on representations which are close to raw sensor data, in contrast to the high-level symbolic representations that abound in the other types of agents discussed so far.

5.5.1 Hypothesis, motivation and benefits

The essential hypothesis of reactive agent-based systems is a specification of the physical-grounding hypothesis, not to be confused with the physical-symbol system hypothesis. The latter hypothesis holds that for a physical system to demonstrate intelligent action it should be a physical-symbol system. The physical-grounding hypothesis challenges this long-held AI view, arguing it is flawed fundamentally, and that it imposes severe limitations on symbolic AI-based systems. This new hypothesis states that in order to build a system that is intelligent, it is necessary to have representations grounded in the physical world. Brooks [40] argues that this hypothesis obviates the need for symbolic representations or models because the world becomes its own best model. Furthermore, this model is always kept up to date since the system is connected to the world via sensors and/or actuators. Hence, the reactive agents hypothesis may be stated as follows: "Smart agent systems can be developed from simple agents which do not have internal symbolic models, and whose 'smartness' derives from the emergent behaviour of the interactions of the various agents."

The key benefits which motivate reactive agents work is the hope that they would be more robust and fault tolerant than other agent-based systems, e.g. a single agent in an ensemble may be lost without any catastrophic effects. Other benefits include flexibility and adaptability in contrast to the inflexibility, slow response times and brittleness of classical AI systems.

5.5.2 Reactive agent applications

There are relatively few reactive software agent-based applications. Partly, due to this reason, there is no standard mode to their operation; rather, they tend to depend on the reactive agent architecture chosen.

Perhaps the most celebrated reactive agent architecture is Brooks' subsumption architecture [41]. The architecture consists of a set of modules, each of which is described in a subsumption language based on augmented finite state

machines (AFSM). An AFSM is triggered into action if its input signal exceeds some threshold, though this is also dependent on the values of suppression and inhibition signals into the AFSM. Note that AFSMs represent the only processing units in the architecture, i.e. there are no symbols as in classical AI work. The modules are grouped and placed in layers which work asynchronously, such that modules in a higher level can inhibit those in lower layers (see Fig 4). Each layer has a hard-wired purpose or behaviour, e.g. to avoid obstacles or to enable/control wandering. This architecture has been used to construct at least ten mobile robots at MIT.

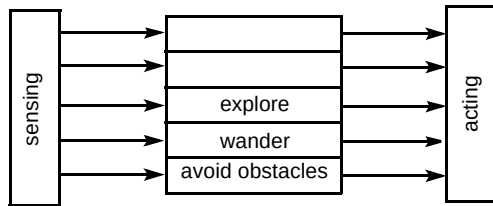


Fig 4 Brooks' subsumption architecture.

Arguably, the most basic reactive architecture is that based on situated-action rules. A situated action agent acts essentially in ways which are 'appropriate' to its situation, where 'situation' refers to a potentially complex combination of internal and external events and states [42]. Situated-action 'agents' have been used in PENGI, a video game [13], and to simulate ant societies where each ant is modelled as an agent, and a limited ecosystem composed of biotapes, shoals of fish and fishermen [14].

5.5.3 A brief critical review of reactive agents work

Many criticisms can be levelled against reactive software agents and their architectures. Firstly, as already noted, there are too few applications about based on them. Secondly, the scope of their applicability is currently limited, mainly games and simulations. To be fair, it is still early days for such research — arguably, symbolic AI did not start delivering any useful industrial applications until more than two decades after it was born. So, there is a clear need to expand the range of languages, theories, architectures and applications for reactive agent-based systems. Thirdly, it is not obvious how to design such systems so that your intended behaviour emerges from the set-up of agents. How many of such agents are required for some applications? Currently, since it is not allowable to tell the agents how to achieve some goal, as with genetic algorithms:

“One has to find a ‘dynamics’, ... involving the system and the environment which will converge towards the desired goal.” [43]

This would not only be time-consuming, but it also smacks of ‘trial and error’ with all its attendant problems.

Furthermore, Maes [43] points out that this situated agents work has some important limitations precisely because “of their lack of explicit goals and goal-handling capabilities”, requiring the designers of the systems to precompile or hard-wire the action selections. Hence, while a planning approach leaves much to the agent, the situated agents approach leaves much to the designers. Fourthly, how are such systems extended, scaled up or debugged? What happens if the ‘environment’ is changed?

5.6 Hybrid agents — an overview

5.6.1 Hypothesis, motivation and benefits

So far, five types of agents have been reviewed — collaborative, interface, mobile, Internet and reactive agents. The debates as to which of them is ‘better’ are academic, sterile and rather premature. Since each type has or promises its own strengths and deficiencies, the trick as always is to maximise the strengths and minimise the deficiencies of the most relevant technique for your particular purpose. Frequently, one way of doing this is to adopt a hybrid approach, like Maes [43], which brought together some of the strengths of both the deliberative and reactive paradigms. In such a case the reactive component, which would take precedence over the deliberative one, brings about the following benefits — robustness, faster response times and adaptability. The deliberative part of the agent would handle the longer term goal-oriented issues. Hence, hybrid agents refer to those whose constitution is a combination of two or more agent philosophies within a singular agent.

5.6.2 Hybrid agent architectures

As is the case with reactive agents, there are few hybrid agent architectures. Typically, however, they have a layered architecture as is evidenced by InteRRaP [44] and TouringMachines [45]. Both are described briefly below.

Muller et al's InteRRaP architecture (Fig 5) comprises three control layers — the behaviour-based layer (BBL), the local planning layer (LPL) and the co-operative planning layer (CPL). The reactive part of the framework is implemented by the BBL which contains a set of situation-action rules. These describe the agent's reactive skills which implement fast situation recognition in order to react to time-critical situations. The intermediate LPL implements local goal-directed behaviour while the topmost CPL enables the agent to plan/co-operate with other agents in order to achieve multi-agent plans, as well as resolve conflicts. LPL and CPL allow for more deliberation. The three layers all work asynchronously with different models in the agent's knowledge base — BBL, LPL and CPL operate with the world, mental and social models respectively. The InteRRaP architecture has been evaluated

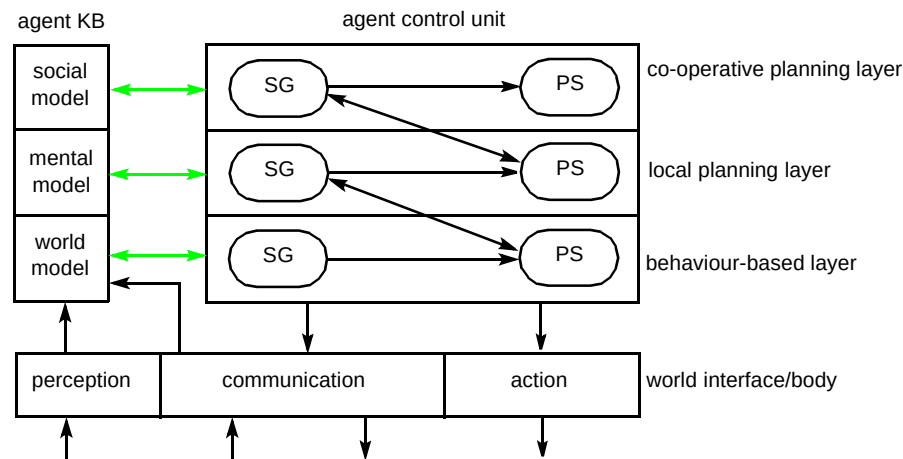


Fig 5 The InteRRaP hybrid architecture (adapted from Fischer et al [46]).

by constructing a FORKS application which simulates forklift robots working in an automated loading dock environment.

Ferguson's TouringMachines [45] architecture, which is similar to Brooks' subsumption architecture (see Fig 4), consists of three control layers — the reactive layer, the planning layer and the modelling layer, which all work concurrently. A key distinction between TouringMachines and Brooks' subsumption architecture on the one hand, and InteRRaP on the other is that the former are horizontal architectures while the latter is a vertical architecture. This means that all the layers in TouringMachines and the subsumption architecture have access to the perception data and can contribute to the actions (as shown in Fig 4), while only the bottom layer in InteRRaP receives and acts on the perceptual data (see Fig 5). Therefore, to achieve co-ordination in TouringMachines, Ferguson has control rules capable of suppressing the input to a certain layer, similar to the inhibition mechanisms in the subsumption architecture.

5.6.3 A brief critical review of hybrid architectures and challenges

Hybrid agent architectures are still relatively few in number but the case for having them is overwhelming. There are usually three typical criticisms of hybrid architectures in general. Firstly, hybridism usually translates to *ad hoc* or unprincipled designs. Secondly, many hybrid architectures tend to be very application-specific, and for good reasons too. Thirdly, the theory which underpins hybrid systems is not usually specified. Therefore, the challenges for hybrid agents research would appear to be quite similar to those identified for reactive agents. In addition to these, hybrids of agent philosophies other than reactive/deliberative ones would be expected to appear.

5.7 Heterogeneous agent systems — an overview

5.7.1 Hypothesis, motivation and benefits

Heterogeneous agent systems refer to an integrated set-up of at least two or more agents which belong to two or more different agent classes. A heterogeneous agent system may also contain hybrid agents. Genesereth and Ketchpel [47] articulate clearly the motivation for heterogeneous agent systems. The essential argument is that the world abounds with a rich diversity of software products. Though these programs work in isolation, there is an increasing demand to have them interoperate, hopefully in such a manner that they provide greater 'added-value' as an ensemble than they do individually. Indeed, a new domain called agent-based software engineering has been invented in order to facilitate the interoperation of miscellaneous software agents.

A key requirement for interoperation among heterogeneous agents is having an agent communication language through which the different software 'agents' can communicate with each other. Genesereth and Ketchpel [47] note that agent-based software engineering is often compared to object-oriented programming in that an agent, like an object, provides a message-based interface to its internal data structures and algorithms. However, they note that there is a key distinction — in object-oriented programming, the meaning of a message may differ from object to object, whereas in agent-based software engineering, agents use a common language with agent-independent semantics. They begin to address the particulars of such an agent-independent communication language through ACL, an agent communication language they have been developing.

5.7.2 How heterogeneous agent systems work

To commence, the rather specific definition is provided of the word ‘agent’ proffered in agent-based software engineering. It defines a software agent as such ‘if and only if it communicates correctly in an agent communication language’ [47]. If new agents are constructed such that they abide by this dictum, then putting them together in a heterogeneous set-up is possible, though not trivial. However, with legacy software, they need to be converted into software agents first. Genesereth and Ketchpel [47] note that there are three ways of doing this conversion. Firstly, the legacy software may be rewritten totally — a most costly approach. Secondly, a transducer approach may be used. The transducer is a separate piece of software which acts as an interpreter between the agent communication language and the legacy software’s native communications protocol. This is the favoured approach in situations where the legacy code may be too delicate to tamper with or is unavailable. Lastly, there is the wrapper technique in which some code is ‘injected’ into the legacy program in order to allow it communicate in ACL. The wrapper can access directly and modify the program’s data structures. This is clearly a more interventionist approach, but offers greater efficiency than the transduction approach.

Once the agents are available, there are two possible architectures to choose from — one in which all the agents handle their own co-ordination or another in which groups of agents can rely on special system programs to achieve co-ordination. The disadvantage of the former is that the communication overhead does not ensure scalability. As a consequence, the federated approach (see Fig 6) is typically preferred.

In the federated set-up of Fig 6, there are five agents distributed on two machines. The agents do not communicate directly with one another but do so through intermediaries called facilitators. Essentially, the agents surrender some of their autonomy to the facilitators who are able to locate other agents on the network capable of providing various services. The facilitators also establish the connections across the environments and ensure correct ‘conversation’ among agents. ARCHON [22] and PACT [48] used such an architecture.

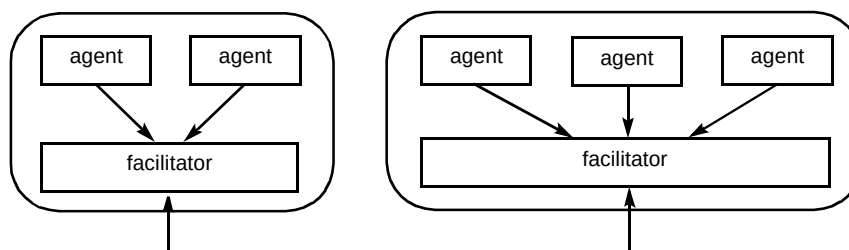


Fig 6 A federated system (adapted from Genesereth and Ketchpel [47]).

The work on heterogeneous agent systems is ongoing and there is a need for methodologies, tools, techniques and standards for achieving such interoperability among heterogeneous information sources.

6. Some general issues

An overview of a broad range of work which goes under the banner of ‘agents’ has been provided, together with their various promises as well as their challenges. We believe, like Greif [49], that agents can have an enormous effect, but that this will appear in everyday products as an evolutionary process. Greif notes, correctly in our view, that agents would initially leverage simpler technologies available in most applications (e.g. word processors, spreadsheets or knowledge-based systems). Then they would gradually be evolved into more complicated applications, doing for example, real work-flow management or controlling real telecommunications networks.

However, apart from the technical issues considered so far, there are also a range of social and ethical problems that are looming, following the large-scale fielding of agent technology, which society would have to grapple with through various legislation.

- Privacy — how do you ensure your agents maintain your privacy when acting on your behalf?
- Responsibility which goes with relinquished authority — when you relinquish some of your responsibility to software agents, be aware of the authority that is being transferred to them. How would you like to come home after a long hard day to find you are the proud owner of a used car negotiated for and bought, courtesy of one of your software agents?
- Legal issues — imagine your agent, which you bought off-the-shelf and customised, offers some bad advice to other peer agents, resulting in liabilities to other people — who is responsible? The company who wrote the agent? You who customised it? Both?
- Ethical issues — already, Eichmann [50] and Etzioni and Weld [37] are concerned enough about the ethics of software agents that they have proposed etiquettes for information service and user agents as they gather information on the WWW.

However, such issues are not that critical immediately, but will become so in the medium to long term.

7. Conclusions

“Smart agents are here to stay. Once unleashed, technologies do not disappear.” [51]

This paper has pilfered from a diverse literature in order to overview the rapidly evolving area of software agents. Only Wooldridge and Jennings [6] have attempted a similar extensive review of this area, which they do from a theories, architecture and languages angle. This paper has overviewed the same area from the viewpoint of the clear diversity of agents being researched in universities and research laboratories world-wide. Its aim has been to provide a useful contribution to understanding this exciting field of software agents. A more detailed exposition of this topic is available in Nwana [52].

8. Acknowledgements

The many informal discussions with Barry Crabtree, Mark Wiegand, Paul O'Brien, Robin Smith and Nader Azarmi, which have shaped some of the views propounded in this paper, are gratefully acknowledged. This work was funded by BT Laboratories.

References

- Hewitt C: 'Viewing Control Structures as Patterns of Passing Messages', *Artificial Intelligence*, 8, No 3, pp 323—364 (1977).
- Huhns M N and Singh M P: 'Distributed Artificial Intelligence for Information Systems', CKBS-94 Tutorial, University of Keele, UK (June 1994).
- Chaib-draa B, Moulin B, Mandiau R and Millot P: 'Trends in Distributed Artificial Intelligence', *Artificial Intelligence Review*, 6, pp 35—66 (1992).
- Bond A H and Gasser L: 'Readings in Distributed Artificial Intelligence', San Mateo, CA, Morgan Kaufmann (1988).
- Gasser L and Huhns M: 'Distributed Artificial Intelligence 2', San Mateo, CA, Morgan Kaufmann (1989).
- Wooldridge M and Jennings N: 'Intelligent Agents: Theory and Practice', *The Knowledge Engineering Review*, 10, No 2, pp 115—152 (1995).
- Wooldridge M and Jennings N (Eds): 'Intelligent Agents, Lecture Notes in Artificial Intelligence', 890, Heidelberg, Springer Verlag (1995).
- Wooldridge M, Mueller J P and Tambe M: 'Intelligent Agents II, Lecture Notes in Artificial Intelligence', 1037, Heidelberg, Springer Verlag (1996).
- Wooldridge M: 'Conceptualising and developing agents', in *Proc UNICOM Seminar on Agent Software*, London, pp 40—54 (April 1995).
- Sycara K: 'Intelligent Agents and the Information Revolution', *UNICOM Seminar on Intelligent Agents and their Business Applications*, London, pp 143—159 (November 1995).
- Guilfoyle C: 'Vendors of Agent Technology', *UNICOM Seminar on Intelligent Agents and their Business Applications*, London, pp 135—142 (November 1995).
- Brooks R A: 'A Robust Layered Control System for a Mobile Robot', *IEEE J Robotics and Automation*, 2, No 1, pp 14—23 (1986).
- Agre P E and Chapman D: 'Pengi: An Implementation of a Theory of Activity', *Proc 6th National Conf on Artificial Intelligence*, San Mateo, CA, Morgan Kaufmann, pp 268—272 (1987).
- Ferber J: 'Simulating with Reactive Agents', in Hillebrand E and Stender J (Eds): 'Many Agent Simulation and Artificial Life', Amsterdam, IOS Press, pp 8—28 (1994).
- Bates J: 'The Role of Emotion in Believable Characters', *Communications of the ACM*, 37, No 7, pp 122—125 (1994).
- Jennings N R: 'Specification and Implementation of a Belief Desire Joint-Intention Architecture for Collaborative Problem Solving', *J Intelligent and Cooperative Information Systems*, 2, No 3, pp 289—318 (1993).
- Rao A S and Georgeff M P: 'BDI Agents: From Theory to Practice', in *Proc 1st Int Conf on Multi-Agent Systems (ICMAS-95)*, San Francisco, USA, pp 312—319 (June 1995).
- Durfee E H, Lesser V R and Corkill D: 'Coherent Cooperation among Communicating Problem Solvers', *IEEE Trans on Computers*, C-36, No 11, pp 1275—1291 (1987).
- URL1: <http://www.cs.cmu.edu/afs/cs.cmu.edu/project/theo-5/www/pleiades.html>
- O'Brien P and Wiegand M: 'Agents of Change in Business Process Management', *BT Technol J*, 14, No 4, pp ... (October 1996)
- Titmuss R, Crabtree I B and Winter C S: 'Agents, mobility and multimedia information', *BT Technol J*, 14, No 4, pp 141—148 (October 1996).
- Wittig T (Ed): 'ARCHON: An Architecture for Multi-Agent Systems', London, Ellis Horwood (1992).
- Maes P: 'Agents that Reduce Work and Information Overload', *Communications of the ACM*, 37, No 7, pp 31—40 (1994).
- Maes P: 'Intelligent Software', *Scientific American*, 273, No 3, (September 1995).
- Kozierok R and Maes P: 'A Learning Interface Agent for Scheduling Meetings', *Proc ACM-SIGCHI Int Workshop on Intelligent User Interfaces*, Florida, pp 81—93 (1993).
- Dent L, Boticario J, McDermott J, Mitchell T and Zabowski D A: 'A Personal Learning Apprentice', in *Proc 10th National Conf on Artificial Intelligence*, San Jose, California, AAAI Press, pp 96—103 (1992).
- Lieberman H: 'Letizia: An Agent that Assists Web Browsing', *Proc IJCAI 95*, AAAI Press (1995).
- Lashkari Y, Metral M and Maes P: 'Collaborative Interface Agents', In *Proc 12th National Conf on Artificial Intelligence*, 1, Seattle, WA, AAAI Press, pp 444—449 (1994).
- Mitchell T, Caruana R, Freitag D, McDermott J and Zabowski D: 'Experience with a Learning Personal Assistant', *Communications of the ACM*, 37, No 7, pp 81—91 (1994).
- URL2: <http://www.genmagic.com>
- Wayner P: 'Agents Unleashed: A Public Domain Look at Agent Technology', Boston, MA, AP Professional (1995).

- 32 Appleby S and Steward S: 'Mobile Software Agents for Control in Telecommunications Networks', *BT Technol J*, 12, No 2, pp 104—113 (April 1994).
- 33 URL3: <http://www.sel.sony.com>
- 34 Plu M: 'Software Agents in Telecommunications Network Environments', UNICOM Seminar on Intelligent Agents and their Business Applications, London, pp 225—243 (November 1995).
- 35 Reinhardt A: 'The Network with Smarts', *Byte*, pp 15—66 (October 1994).
- 36 Wayner P: 'Free Agents', *Byte*, 105—114 (March 1995).
- 37 Etzioni O and Weld D: 'A Softbot-Based Interface to the Internet', *Communications of the ACM*, 37, No 7, pp 72—76 (1994).
- 38 Davies N J and Weeks R: 'Jasper: Communicating Information Agents', in *Proc 4th Int Conf on the World Wide Web*, Boston, USA (December 1995).
- 39 Maes P (Ed): 'Designing Autonomous Agents: Theory and Practice from Biology to Engineering and Back', London, The MIT Press (1991).
- 40 Brooks R A: 'Elephants Don't Play Chess', in Maes P (Ed): 'Designing Autonomous Agents: Theory and Practice from Biology to Engineering and Back', London, MIT Press, pp 3—15 (1991).
- 41 Brooks R A: 'Intelligence without Representation', *Artificial Intelligence*, 47, pp 139—159 (1991).
- 42 Connah D: 'The Design of Interacting Agents for Use in Interfaces', in Brouwer-Janse D and Harrington T L (Eds): 'Human/Machine Communication for Educational Systems Design', NATO ASI Series, Series F, Computer and Systems Sciences, 129, Heidelberg, Springer Verlag (1994).
- 43 Maes P: 'Situated Agents Can Have Goals', in Maes P (Ed): 'Designing Autonomous Agents: Theory and Practice from Biology to Engineering and Back', London, The MIT Press, pp 49—70 (1991).
- 44 Muller J P, Pishel M and Thiel M: 'Modelling Reactive Behaviour in Vertically Layered Agent Architectures', in Wooldridge M and Jennings N (Eds): 'Intelligent Agents, Lecture Notes in Artificial Intelligence 890', Heidelberg, Springer Verlag, pp 261—276 (1995).
- 45 Ferguson I A: 'Towards an Architecture for Adaptive, Rational, Mobile Agents', in Werner E and Demazeau Y (Eds): 'Decentralized AI 3: Proc 3rd European Workshop on Modelling Autonomous Agents and Multi-Agent Worlds (MAAMAW-91)', Amsterdam, Elsevier, pp 249—262 (1992).
- 46 Fischer K, Muller J P and Pischel M: 'Unifying Control in a Layered Agent Architecture', Technical Report TM-94-05, German Research Center for AI (DFKI GmbH) (1996).
- 47 Genesereth M R and Ketchpel S P: 'Software Agents', *Communications of the ACM*, 37, No 7, pp 48—53 (1994).
- 48 Cutkosky M R, Englemore R S, Fikes R E, Genesereth M R, Gruber T R, Tenenbaum J M and Weber J C: 'PACT: An Experiment in Integrating Concurrent Engineering Systems', *IEEE Computer*, 1, pp 28—37 (January 1993).
- 49 Greif I: 'Desktop Agents in Group-Enabled Products', *Communications of the ACM*, 37, No 7, pp 100—105 (1994).
- 50 Eichmann D T: 'Ethical Web Agents', HREF = "ethics.html#FN1", <http://www.ncsa.uiuc.edu/SDG/IT94/P...ents/eichmann.ethical/eichman.htm> (1994).
- 51 Norman D: 'How might people interact with agents', *Communications of the ACM*, 37, No 7, pp 68—76 (1994).
- 52 Nwana H S: 'Software Agents: an overview', *Knowledge Engineering Review*, 11, No 3 (1996).



Hyacinth S Nwana holds a BSc in Computer Science and Electronic Engineering (1985), an MSc in Computer Science (1986) and a PhD in the area of Artificial Intelligence (1989). Between 1989 and 1995, he worked in various roles (lecturer, researcher, contract and visiting scientist) for the Universities of Liverpool, Keele, Calgary (sponsored by the Royal Society), Shell Research Laboratories and BT Laboratories. He joined BT in October 1995 and leads part of BT's software agents research programme, as well as a team researching into the engineering of agent-based systems and is contactable on e-mail at:

hyacinth@info.bt.co.uk. He has a broad range of AI-related interests and has written on software agents, AI and education, distributed AI, machine learning, neural networks, knowledge acquisition, computer science education and cognitive science. In 1991, he principally won the Digital Equipment Corporation (DEC) European Artificial Intelligence prize. He is currently pursuing a part-time MEd degree at Cambridge. He is a member of the British Computer Society and a Chartered Engineer.



Divine Ndumu graduated with first class honours in Civil Engineering from Imperial College and was winner of the Imperial College Governor's prize for excellence in Civil Engineering in 1989. He also graduated with first class honours in Mathematics and Computing from the Open University in 1991. In 1993 he was awarded a PhD in Structural Engineering Applications of Artificial Intelligence from Imperial College, University of London. In 1992 he joined the University of Central Lancashire as a lecturer, and is currently serving a one year Research Fellowship secondment at BT Laboratories. The focus of his research is on the practical applications of AI techniques to real-world problems.